



CSE471: System Analysis and Design
Project Report
Project Title: “BiteExpress” Online Food Delivery Website

Group No: 05, CSE471 Lab Section: 06, Semester: Summer 2024	
ID	Name
21301381	Kazi Rafiul Kader
21201402	Nazmus Sakib
19101560	Faiyaaz Ismail
21101015	Tasnim Hasan Rosan

Table of contents

Section No	Content	Page No.
1	Introduction	3
2	Functional Requirements	4-5
3	User Manual	6-16
4	Frontend Development	17-29
5	Backend Development	29-43
6	Technology(Framework, Languages)	44
7	Github Repo Link	44
8	Individual Contribution	44

Introduction

The project is for an online food delivery system. We named it BiteExpress. This project is focused on developing a user-friendly and secure food ordering platform that caters to both customers and restaurant owners.

The system offers a comprehensive account management experience, allowing users to create accounts as customers or restaurant owners using their email or phone number. Key features include an easy login process, password reset, and profile management for updating personal information.

For customers, the platform provides a rich food menu browsing experience, complete with categories, item descriptions, and prices. Users can search for specific items or restaurants and customize their orders by adjusting ingredients and portion sizes. The cart management feature ensures that users can easily add, modify, or remove items before checkout, making the ordering process smooth and efficient.

On the restaurant side, owners can manage and update their menus with ease, ensuring that their offerings are always up-to-date. The project aims to deliver a seamless and intuitive experience for both customers and restaurant owners, promoting a secure and efficient online food ordering ecosystem.

Online food ordering systems have revolutionized the way customers interact with restaurants. With the rise of technology and mobile accessibility, these platforms have become essential for modern dining experiences. Customers can now browse menus, customize orders, and make payments without ever leaving their homes. This convenience has not only boosted customer satisfaction but also expanded restaurant reach beyond physical locations, allowing them to serve more customers at any time. Online ordering has become an integral part of the food industry, offering flexibility, speed, and ease to customers who prefer to dine in the comfort of their homes or workplaces.

Additionally, online food delivery systems provide restaurants with a new channel for increasing revenue. By embracing digital menus and order management systems, restaurant owners can optimize their services, track customer preferences, and streamline operations. The data collected through online orders offers valuable insights into customer behavior, helping restaurants improve their menus, pricing, and promotions. This not only strengthens customer relationships but also ensures that restaurants stay competitive in an increasingly digital market. As online food delivery continues to grow, it creates opportunities for businesses to innovate and meet the evolving needs of today's consumers.

Functional Requirements

1. User Registration and Authentication

1.1 Account Creation

The system will allow users to create a new account using their email, phone number.(There will be two types of account Customer, Restaurant owner)

1.2 Login

The system will enable users to log in using their credentials, such as email/phone number and password.

1.3 Password Reset

The system will allow users to reset their passwords via email.

1.4 Profile Management

The system will enable users to view and update their personal information, including name, address, and contact details.

2. Food Menu and Ordering

2.1 Browsing the Menu

The system will allow users to browse the food menu, including categories, item descriptions, and prices.

2.2 Search Functionality

The system will enable users to search for specific food items or categories or restaurants.

2.3 Order Customization

The system will allow users to customize their orders, such as adding/removing ingredients and choosing portion sizes.

2.4 Cart Management

The system will enable users to add, remove, or modify items in their cart before checkout.

2.5 Order Placement

The system will allow users to review their order details, including items, quantities, and total cost, before placing the order.

2.6 Menu Management

The system will allow Restaurant owners to update their menu.

User Manual

1) The user will first see this Landing Page:

BiteExpress

LoginRegistration



How to Place an Order

Ordering your favorite food is easy! Choose a restaurant, browse the menu, select your items, and proceed to checkout. Need assistance? Our customer support team is here to help.

Payment Options

Why Choose Us?

We are committed to providing you with the best food delivery experience. Here's why you should choose us:

- Wide variety of restaurants and cuisines
- Easy and user-friendly ordering process
- Secure and multiple payment options
- Real-time order tracking
- Fast and reliable delivery



List your restaurant or shop on BiteExpress

Would you like millions of new customers to enjoy your amazing food and groceries? So would we!

It's simple: we list your menu and product lists online, help you process orders, pick them up, and deliver them to hungry customers – in a heartbeat!

Interested? Let's start our partnership today!

Get Started!

We are available in:



Dhaka



Chittagong



Bagerhat

Join BiteExpress as a Delivery Partner!

Are you looking for flexible work opportunities?
Become a delivery partner with BiteExpress and start earning today!

Deliver food and groceries to customers in your area.
We handle the orders and logistics; you enjoy the freedom to choose your hours.

Interested? Let's get you started on our platform!

[Get Started!](#)



BiteExpress Bangladesh

Are you hungry? Did you have a long and stressful day? Interested in getting a cheesy pizza delivered to your office or looking to avoid the weekly shop? Then BiteExpress Bangladesh is the right destination for you! BiteExpress offers you a long and detailed list of the best restaurants and shops near you to help make your everyday easier.

Our online food delivery service has it all, whether you fancy a juicy **burger** from Takeout, fresh sushi from Samdado or peri peri chicken from Nando's, BiteExpress Bangladesh has over 2000 restaurants available from Dhaka to Chittagong through to Sylhet.

Did you know you can order your groceries and more from BiteExpress, too? Check out BiteExpress shops for favourite partners like Unimart, Suborno, Shwapno, Bengal Meat, and much more. Sit back and relax – let BiteExpress Bangladesh take the pressure off your shoulders.

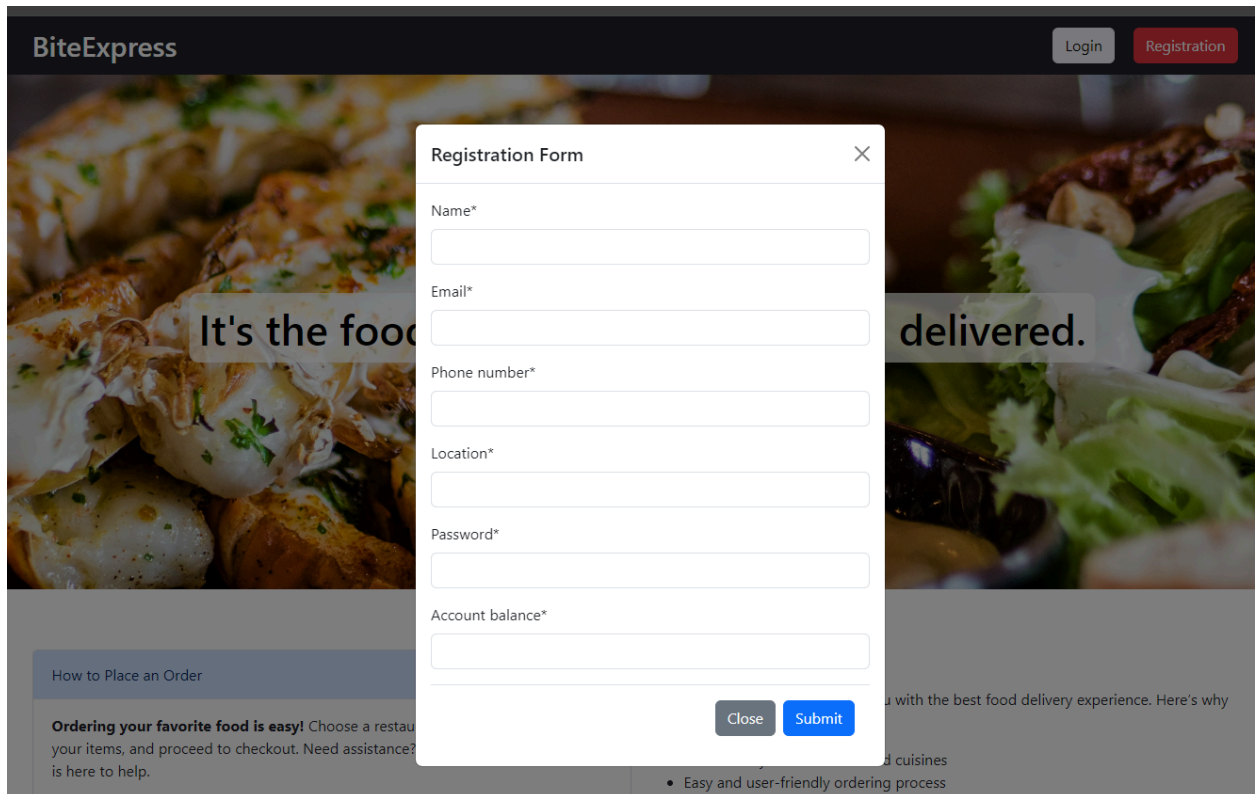
[Home](#) [Features](#) [Pricing](#) [FAQs](#) [About](#)

© 2024 Company, Inc

2) User's can use this to Login to their accounts:

A screenshot of a website with a food-themed background. A white login form is overlaid in the center. The form has a title 'Login Form' with a close button (X) in the top right corner. It contains two input fields: 'Email' with the text 'rafi' and 'Password' with four dots. Below the password field are three buttons: a blue 'Login' button, a grey 'Close' button, and a blue link 'Forget Password?'. At the bottom of the page, there is a section titled 'Why Choose Us?' with a sub-header 'We are committed to providing you with the best food delivery e' and a paragraph 'you should choose us:'. To the left of this section, there is a small upward arrow icon and a line of text 'd is easy! Choose a restaurant, browse the menu, select'.

3) Buyer Registration:



The image shows a screenshot of the BiteExpress website. At the top, there is a dark header with the "BiteExpress" logo on the left and "Login" and "Registration" buttons on the right. The main background features a large image of food with the text "It's the food delivered." overlaid. A white "Registration Form" modal is centered on the screen. The form includes fields for Name*, Email*, Phone number*, Location*, Password*, and Account balance*. At the bottom of the form are "Close" and "Submit" buttons. In the bottom left corner, there is a section titled "How to Place an Order" with a brief description of the ordering process.

Registration Form

Name*

Email*

Phone number*

Location*

Password*

Account balance*

Close Submit

How to Place an Order

Ordering your favorite food is easy! Choose a restaurant, add items to your cart, and proceed to checkout. Need assistance? Our support team is here to help.

4) **Restaurant Registration:** The registration of restaurant owners and buyers are different. So it does not cause confusion for the users.



List your restaurant or shop on BiteExpress

Would you like millions of new customers to enjoy your amazing food and groceries? So would we!

It's simple: we list your menu and product lists online, help you process orders, pick them up, and deliver them to hungry customers – in a heartbeat!

Interested? Let's start our partnership today!

Get Started!

Restaurant Business Registration

Name*

Email*

Phone number*

Location*

Password*

Description*

Submit

5) **Delivery personnel Registration:** Delivery personnel need to register in this form.

Join BiteExpress as a Delivery Partner!

Are you looking for flexible work opportunities?
Become a delivery partner with BiteExpress and start earning today!

Deliver food and groceries to customers in your area.
We handle the orders and logistics; you enjoy the freedom to choose your hours.

Interested? Let's get you started on our platform!

[Get Started!](#)



Delivery Man Registration

Name*

Email*

Phone number*

Location*

Password*

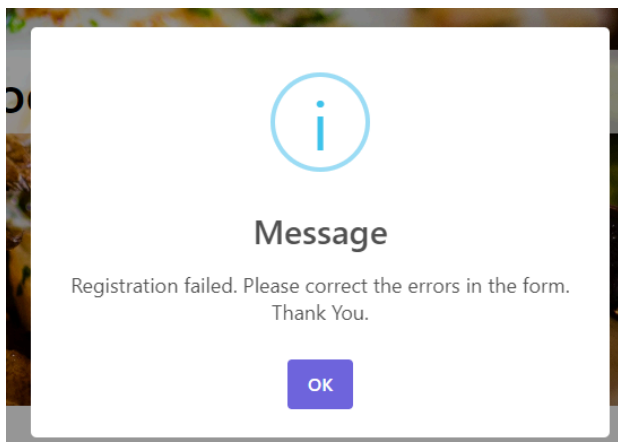
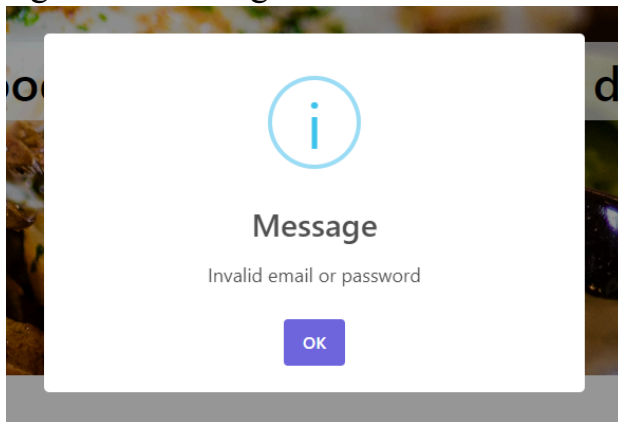
Vehicle number*

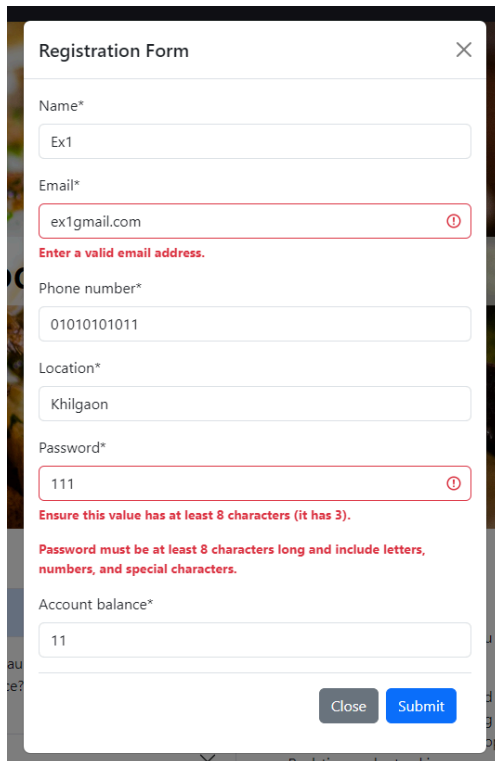
Submit

[Home](#) [Features](#) [Pricing](#) [FAQs](#) [About](#)

© 2024 Company, Inc

6) **Login and Registration Authentication:** User's need to enter valid data for registration or login in the website.





Registration Form

Name*
Ex1

Email*
ex1gmail.com ⓘ
Enter a valid email address.

Phone number*
01010101011

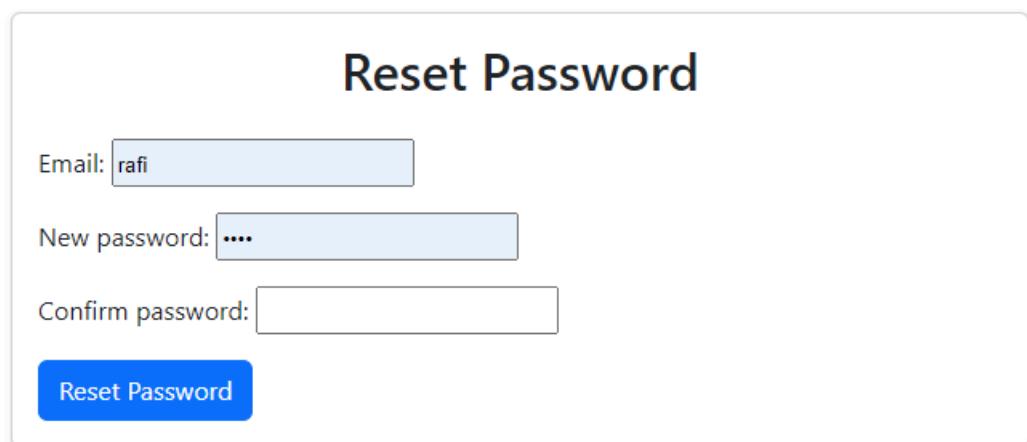
Location*
Khilgaon

Password*
111 ⓘ
Ensure this value has at least 8 characters (it has 3).
Password must be at least 8 characters long and include letters, numbers, and special characters.

Account balance*
11

Close Submit

- 7) **Forget Password:** If the user forgets password. They can reset their password, giving their used email and confirming their password again.



Reset Password

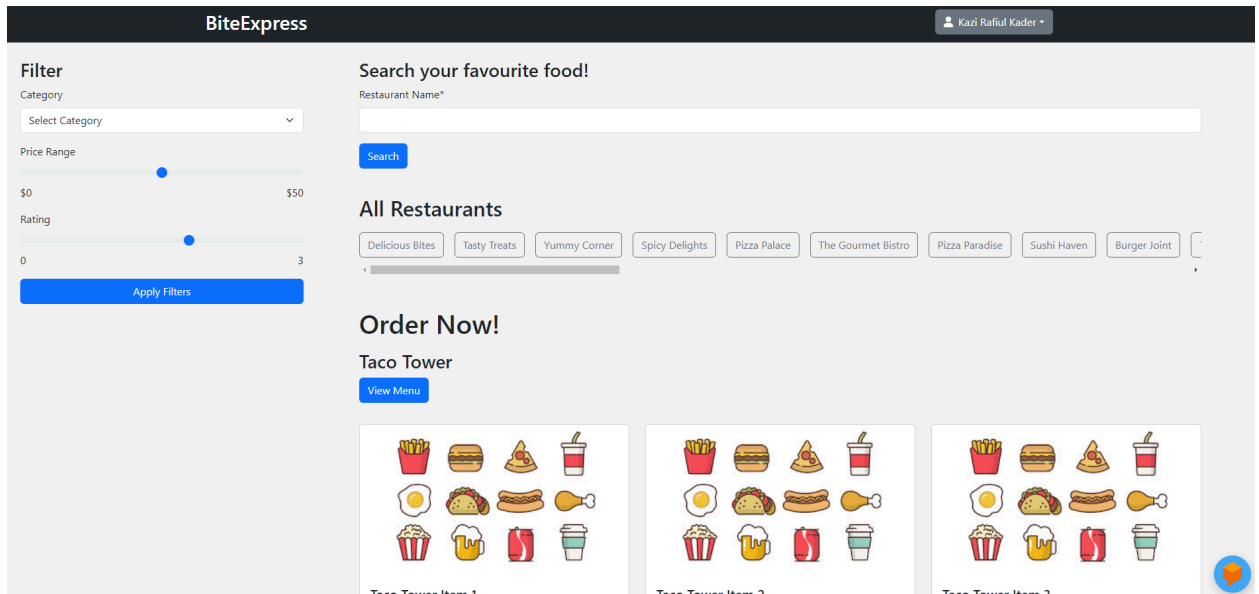
Email: rafi

New password:

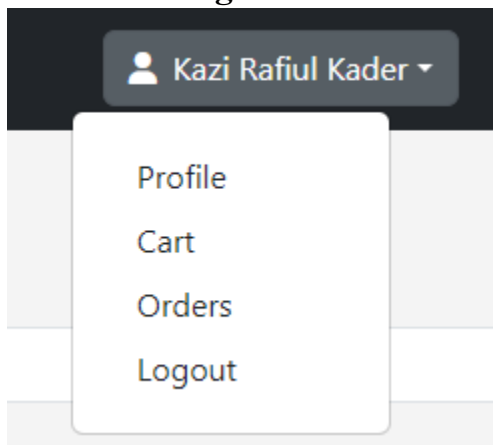
Confirm password:

Reset Password

- 8) **Buyer Dashboard:** Buyer's can use their dashboard for ordering food and searching restaurant's using their name.



- 9) Using dropdown to change profile information, accessing their carts, orders and logout from their account.



- 10) **Changing profile information:** Any user can change their profile informations

BiteExpress

Kazi Rafiul Kader

Name*

Kazi Rafiul Kader

Email*

exmp1@gmail.com

Phone number*

019192388776

Location*

Khilgaon

Password*

hello

Account balance*

1000.00

Submit

11) Searching by Food name or Restaurant name: Buyers can use restaurant name and food name to search.

Results for "Brownies with ice-cream"

Delicious Bites

Brownies with ice-cream

Delicious Bites

Tasty Treats

Brownies with ice-cream

Tasty Treats

BiteExpress

Kazi Rafiul Kader

Delicious Bites's Menu

Cheeseburger

Juicy beef patty with melted cheese, lettuce, and tomato.

Price: \$9.99

0

Add to Cart

Chicken Wings

Crispy fried chicken wings served with dipping sauce.

Price: \$7.99

0

Add to Cart

Fish and Chips

Crispy battered fish served with fries and tartar sauce.

Price: \$10.99

0

Add to Cart

Caesar Salad

Fresh romaine lettuce with Caesar dressing, croutons, and Parmesan cheese.

Price: \$8.99

0

Add to Cart

Margarita Pizza

Classic pizza with tomato sauce, mozzarella, and fresh basil.

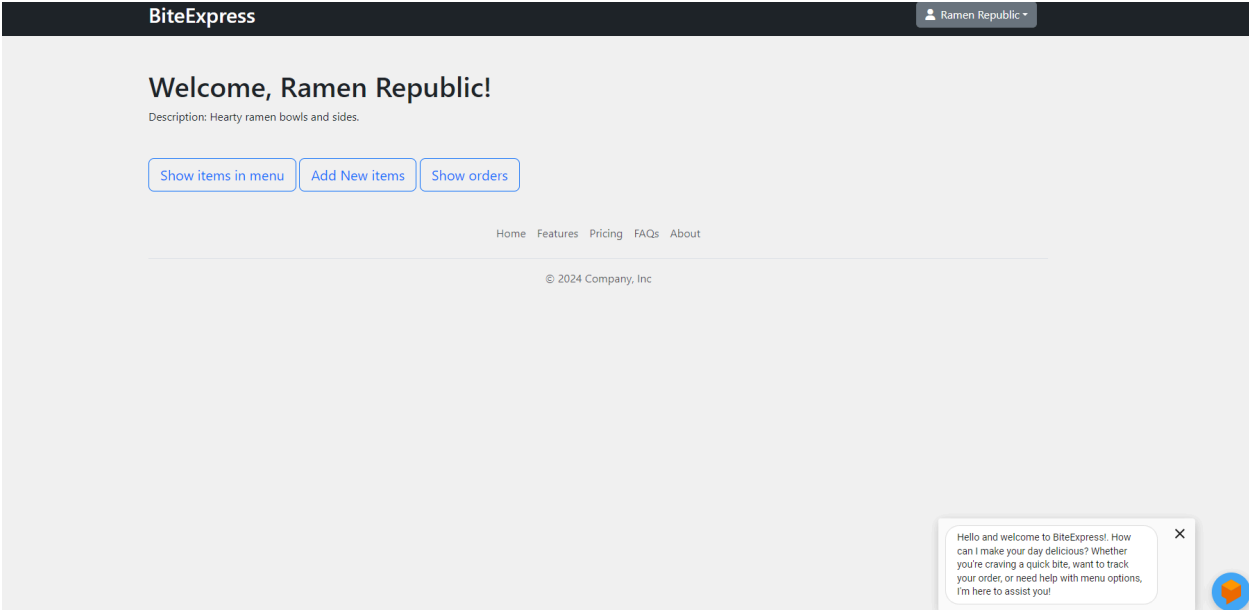
Price: \$12.99

0

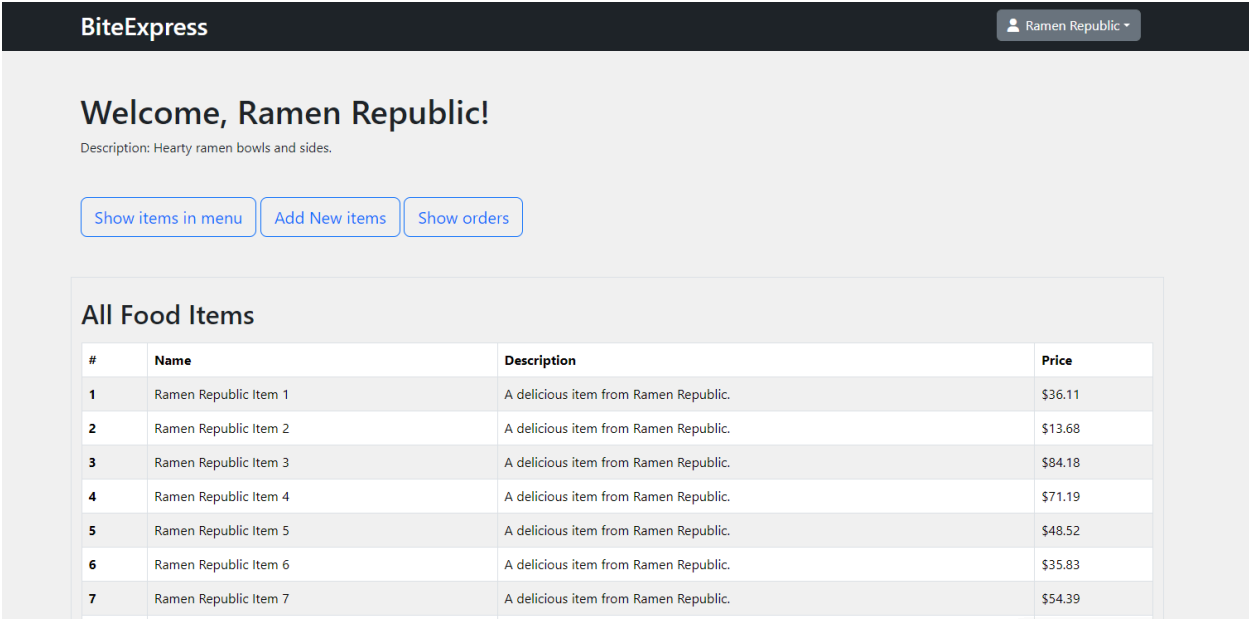
Add to Cart

Chocolate Brownie

12) Restaurant owner dashboard: After logging in, restaurant owners will see this dashboard and use the features.



13) Showing items in their menu if they click “Show items in menu” button



14) Adding items in their menu if they click “Add New items” button

Welcome, Ramen Republic!

Description: Hearty ramen bowls and sides.

[Show items in menu](#) [Add New items](#) [Show orders](#)

Name*

Price*

Description*

[Submit](#)

[Home](#) [Features](#) [Pricing](#) [FAQs](#) [About](#)

Hello and welcome to BiteExpress! How can I make your day delicious? Whether you're craving a quick bite, want to track

15) Showing the current and previous order of the restaurant when they click “Show Orders”

Welcome, Tasty Treats!

Description: Your go-to place for tasty treats and snacks.

[Show items in menu](#) [Add New items](#) [Show orders](#)

All Orders

Order #	Total Amount	Status	Action
10	\$3.99	COMPLETED	Completed
8	\$3.99	COMPLETED	Completed
4	\$15.96	COMPLETED	Completed
3	\$15.96	COMPLETED	Completed
2	\$43.91	COMPLETED	Completed
1	\$31.92	COMPLETED	Completed

16) Buyer order dashboard : Buyers need to use the dropdown button of “Orders” to see their order and previous order history.

Your Orders
Order #11 - Total: \$455.55 Status: COMPLETED
Order #10 - Total: \$3.99 Status: COMPLETED
Order #9 - Total: \$30.61 Status: PENDING
Order #7 - Total: \$19.98 Status: COMPLETED

- 17) **Buyer cart dashboard:** For order customization buyers can use their cart option so they can add, reduce items and confirm their order.

Cart Cart Items <table> <tr> <td> Ice Cream Sundae Quantity: 9 Price: \$53.91 </td> <td>Remove</td> </tr> <tr> <td> French Fries Quantity: 1 Price: \$3.99 </td> <td>Remove</td> </tr> <tr> <td> Onion Rings Quantity: 1 Price: \$4.99 </td> <td>Remove</td> </tr> </table>	Ice Cream Sundae Quantity: 9 Price: \$53.91	Remove	French Fries Quantity: 1 Price: \$3.99	Remove	Onion Rings Quantity: 1 Price: \$4.99	Remove	Add more items Review Order
Ice Cream Sundae Quantity: 9 Price: \$53.91	Remove						
French Fries Quantity: 1 Price: \$3.99	Remove						
Onion Rings Quantity: 1 Price: \$4.99	Remove						

- 18) **Pop up for review order:** This is to make sure the buyer is ordering the right thing before confirmation.

Order Summary

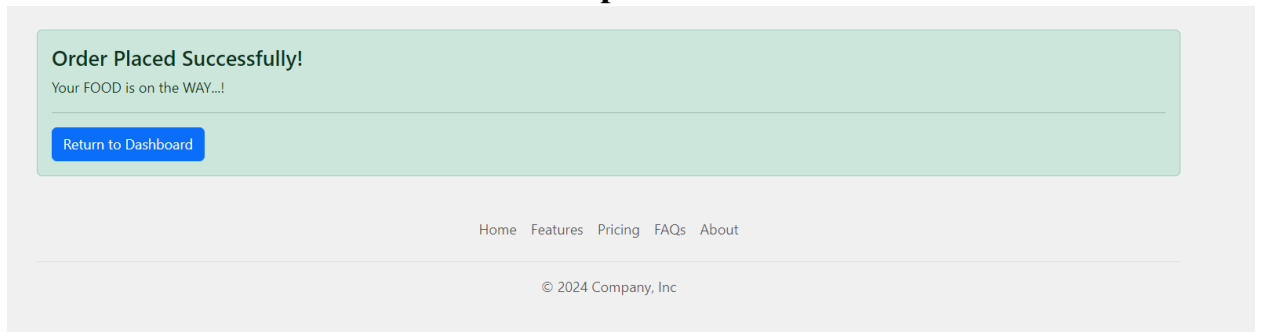
Ice Cream Sundae Quantity: 9 Total: \$53.91
French Fries Quantity: 1 Total: \$3.99
Onion Rings Quantity: 1 Total: \$4.99

Total: \$62.89

Close

Place Order

19) Confirmation for successful order placement



Frontend Development

1) The following codes are for landing page:

```
<!doctype html>
<html lang="en">
<head>
  (% load static %)
  <meta charset="utf-8">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/sweetalert2@11/dist/sweetalert2.min.css">
  <script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>

  <style>
    body {
      background-color: #f4f4f4;
      margin: 0; /* Ensure no default margins */
      padding: 0; /* Ensure no default padding */
    }

    .half-screen-height {
      height: 60vh; /* Adjust height as needed */
      width: 100%;
      object-fit: cover;
    }

    .bg-image-container {
      position: relative;
      height: 60vh; /* Same as .half-screen-height height */
    }

    .text-overlay {
      position: absolute;
      top: 50%;
      left: 50%;
      transform: translate(-50%, -50%);
      text-align: center;
      width: 100%; /* Ensure full width */
      padding: 0 20px; /* Adjust padding as needed */
    }

    .landing-font {
      color: black;
      font-size: 45px;
      background-color: rgba(255, 255, 255, 0.7);
      border-radius: 10px;
      padding: 10px;
      display: inline-block; /* Ensures the background doesn't stretch full width */
    }
  </style>

  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>BiteExpress</title>
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/animate.css/4.1.1/animate.min.css">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-QWTKZyjpPEjISV5WaRU90FeRpok6YctnVmDrsPnlyT2BtjXh0JHjV9HwaALEuH" crossorigin="anonym">
</head>
<body>
  <nav class="navbar navbar-dark bg-dark navbar-expand-lg px-3">
    <div class="container">
      <a href="{% url 'register-user' %}" class="navbar-brand mb-0 h1" id="be">
        {% block brand %}BiteExpress{% endblock %}
      </a>
      <div class="ms-auto">
        <button type="button" class="btn btn-light" data-bs-toggle="modal" data-bs-target="#loginModal">
          Login
        </button>
        <button type="button" class="btn btn-danger mx-3" data-bs-toggle="modal" data-bs-target="#registrationModal">
          Registration
        </button>
      </div>
    </div>
  </nav>

  {% if messages %}
  <script type="text/javascript">
    document.addEventListener("DOMContentLoaded", function() {
      {% for message in messages %}
      Swal.fire({
        title: 'Message',
        text: '{{ message }}',
        icon: 'info',
        confirmButtonText: 'OK'
      });
      {% endfor %}
    });
  </script>
  {% endif %}

  <div class="container-fluid p-0 bg-image-container" id="landing">
    
    <div class="text-overlay">
      <h1 class="landing-font">It's the food and groceries you love, delivered.</h1>
    </div>
  </div>
```

2) Login for buyer/restaurant owner/ delivery person Code :

```
</style>
<div class="modal fade" id="loginModal" tabindex="-1" aria-labelledby="loginModalLabel" aria-hidden="true">
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title" id="loginModalLabel">Login Form</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
      </div>
      <div class="modal-body">
        <form id="loginForm" action="{% url 'login_user' %}" method="post">
          {% csrf_token %}
          <div class="mb-3">
            <label for="email" class="form-label">Email</label>
            <input type="email" class="form-control" id="email" name="email" required>
          </div>
          <div class="mb-3">
            <label for="password" class="form-label">Password</label>
            <input type="password" class="form-control" id="password" name="password" required>
          </div>
          <div class="modal-footer">
            <a href="{% url 'forget_password' %}">Forget Password?</a>
            <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Close</button>
            <button type="submit" class="btn btn-primary">Login</button>
          </div>
        </form>
      </div>
    </div>
  </div>
</div>
```

3) Registration for buyer:

```
<div class="modal fade" id="registrationModal" tabindex="-1" aria-labelledby="registrationModalLabel" aria-hidden="true">
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title" id="registrationModalLabel">Registration Form</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
      </div>
      <div class="modal-body">
        {% load crispy_forms_tags %}
        <form id="registrationForm" action="{% url 'register_user' %}" method="post" novalidate>
          {% csrf_token %}
          {{ form|crispy }}
          <div class="modal-footer">
            <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Close</button>
            <button type="submit" class="btn btn-primary">Submit</button>
          </div>
        </form>
      </div>
    </div>
  </div>
</div>
```

4) Registration for restaurant Owner:

```

{% extends 'members/base.html' %}
{% load crispy_forms_tags %}

{% block content %}
<div class="container mt-5">
  <h1 class="text-center mb-4">Restaurant Business Registration</h1>
  <div class="row justify-content-center">
    <div class="col-lg-6 col-md-8 col-12">
      <div class="form-group">
        <form action="{% url 'restaurant_registration' %}" method="post">
          {% csrf_token %}
          {{ form|crispy }}
          <button type="submit" class="btn btn-primary w-100">Submit</button>
        </form>
      </div>
    </div>
  </div>
</div>
{% endblock %}

```

5) Registration for delivery personnel:

```

{% extends 'members/base.html' %}
{% load crispy_forms_tags %}
{% block content %}
<div class="container">
  <h1>Delivery Man Registration</h1>
  <div class="form-group">
    <form action="{% url 'delivery_registration' %}" method="post">
      {% csrf_token %}
      {{ form|crispy }}
      <button type="submit" class="btn btn-primary">Submit</button>
    </form>
  </div>
</div>

{% endblock %}

```


6) Buyer Dashboard Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  {% load static %}
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{% block title %}BiteExpress{% endblock %}</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnVmDrsNlyT2bRjXh03Mh3Y6hWHALEwIH" crossorigin="anonymous">
  <link rel="stylesheet" href="{% static 'members/css/style.css' %}">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min.css">
  <style>...
</style>
</head>
</head>
<body>

<nav class="navbar navbar-dark bg-dark navbar-expand-lg px-3">
  <div class="container">
    <a href="{% url 'dashboard' %}" class="navbar-brand mb-0 h1" id="be">
      {% block brand %}BiteExpress{% endblock %}
    </a>
    <div class="dropdown px-3">
      <button class="btn btn-secondary dropdown-toggle d-flex align-items-center" type="button" data-bs-toggle="dropdown" aria-expanded="false">
        <i class="fas fa-user me-2"></i>
        {{ user_name }}
      </button>
      <ul class="dropdown-menu p-3">
        <li><a class="dropdown-item" href="{% url 'profile' %}">Profile</a></li>
        <li><a class="dropdown-item" href="#">Cart</a></li>
        <li><a class="dropdown-item" href="#">Orders</a></li>
        <li><a class="dropdown-item" href="{% url 'register_user' %}">Logout</a></li>
      </ul>
    </div>
  </div>
</nav>

{% if messages %}
  {% for message in messages %}
    <div class="d-flex justify-content-center">
      <div class="alert alert-primary alert-dismissible fade show custom-alert col-12 col-md-6" role="alert">
        {{ message }}
        <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
      </div>
    </div>
  {% endfor %}
{% endif %}

{% block content %}
```

7) Buyer dashboard Filter Code:

```
<div class="container mt-4">
  <h3>Filter</h3>
  <div class="mb-3">
    <label for="categorySelect" class="form-label">Category</label>
    <select id="categorySelect" class="form-select">
      <option value="">Select Category</option>
      <option value="fastfood">Fast Food</option>
      <option value="desserts">Desserts</option>
      <option value="beverages">Beverages</option>
    </select>
  </div>
  <div class="mb-3">
    <label for="priceRange" class="form-label">Price Range</label>
    <input type="range" class="form-range" id="priceRange" min="0" max="100" step="5" oninput="updatePriceValue(this.value)">
    <div class="d-flex justify-content-between">
      <span>${0}</span>
      <span id="priceValue">${50}</span>
    </div>
  </div>
  <div class="mb-3">
    <label for="ratingRange" class="form-label">Rating</label>
    <input type="range" class="form-range" id="ratingRange" min="0" max="5" step="1" oninput="updateRatingValue(this.value)">
    <div class="d-flex justify-content-between">
      <span>0</span>
      <span id="ratingValue">3</span>
    </div>
  </div>
  <div class="mt-3">
    <button type="button" class="btn btn-primary w-100">Apply Filters</button>
  </div>
</div>
</div>
```

8) Search option Code and search result show code:

```
<div class="col-lg-9 col-md-8 col-sm-12">
  <div class="container mx-4" id="main-buyer-container">
    {% block main_content %}
      <div class="container my-4">
        <h3>Search your favourite food!</h3>
        <form method="post">
          {% csrf_token %}
          {{ form|crispy }}
          <button type="submit" class="btn btn-primary mb-3">Search</button>
        </form>
      </div>
      <div class="container mb-5">
        <h2>All Restaurants</h2>
```

```
{% extends 'dashboard/base_dashboard.html' %}
{% block content %}
<div class="container my-4">
  <h3 class="mb-4">Results for "{{ search_query }}"</h3>
  <div class="row">
    {% for restaurant_name, items in restaurant_items.items %}
      <div class="col-lg-6 col-md-12 mb-4">
        <div class="card shadow-sm">
          <div class="card-body">
            <h5 class="card-title">{{ restaurant_name }}</h5>
            <ul class="list-group list-group-flush">
              {% for item in items %}
                <li class="list-group-item d-flex justify-content-between align-items-center">
                  {{ item.name }}
                  <form action="{% url 'restaurant_menu' user_name=user_name restaurant_name=restaurant_name %}" method="get">
                    <button type="submit" class="btn btn-sm btn-primary">{{ restaurant_name }}</button>
                  </form>
                </li>
              {% endfor %}
            </ul>
          </div>
        </div>
      </div>
    {% endfor %}
  </div>
</div>
{% endblock %}
```

```
{% extends 'dashboard/base_dashboard.html' %}
{% block content %}
<div class="container my-4">
  <h1>{{ restaurant_name }}'s Menu</h1>
  <ul class="list-group">
    {% for item in menu_items %}
      <li class="list-group-item d-flex justify-content-between align-items-center">
        <div>
          <h5>{{ item.name }}</h5>
          <p>{{ item.description }}</p>
          <p><strong>Price:</strong> ${{ item.price }}</p>
        </div>
        <div class="d-flex align-items-center">
          <div class="input-group me-3" style="width: 120px;">
            <label for="quantity_{{ item.id }}" class="visually-hidden">Quantity</label>
            <input type="number" id="quantity_{{ item.id }}" class="form-control" value="0" min="1">
          </div>
          <button type="button" class="btn btn-primary add-to-cart" data-item-id="{{ item.id }}">Add to Cart</button>
        </div>
      </li>
    {% endfor %}
  </ul>
</div>

<script>
  document.querySelectorAll('.add-to-cart').forEach(button => {
    button.addEventListener('click', function() {
      const itemId = this.getAttribute('data-item-id');
      const quantity = document.getElementById('quantity_' + itemId).value;
      alert('Added $(quantity) of item $(itemId) to cart.');
```

9) All restaurant names accordion slider code:

```
</div>
</div>
<div class="container mb-5">
  <h2>All Restaurants</h2>
  <div class="scrollable-container">
    <ul class="restaurant-list">
      {% for restaurant in all_restaurants %}
        <button class="btn btn-outline-secondary me-3"><li>{{ restaurant }}</li></button>
      {% endfor %}
    </ul>
  </div>
</div>
```

10) Random restaurant Show in Buyer dashboard:

```
<div class="col">
  <h1 class="mb-3">Order Now!</h1>
  {% for restaurant_name, items in restaurant_items.items %}
    <div class="card-header mb-3">
      <h3 id="restaurant_name">{{ restaurant_name }}</h3>
      <button class="btn btn-primary mb-3">
        <a href="{% url 'restaurant_menu' user_name=user_name restaurant_name=restaurant_name %}" style="text-decoration: none; color: inherit;">View Menu</a>
      </button>
    </div>
    <div class="card-body mb-3">
      <div class="row">
        {% for item in items %}
          <div class="col-lg-4 col-md-6 mb-4">
            <div class="card">
              
              <div class="card-body">
                <h5 class="card-title">{{ item.name }}</h5>
                <p class="card-text">{{ item.description }}</p>
                <p class="card-text"><strong>Price:</strong> ${{ item.price }}</p>
              </div>
            </div>
          </div>
        {% endfor %}
      </div>
    </div>
  </div>
{% endfor %}
```

11) Customer/Restaurant owner/Delivery man profile change:

```
press > dashboard > templates > dashboard > <> user_profile.html > <> div.container > <> form > <> button.btn.
{% extends 'dashboard/base_dashboard.html' %}
{% load crispy_forms_tags %}

{% block content %}
<div class="container">
  <form action="{% url 'profile' %}" method="post">
    {% csrf_token %}
    {{ form|crispy }}
    <button type="submit" class="btn btn-primary">Submit</button>
  </form>
</div>
{% endblock %}
```

12) Showing items in menu restaurant dashboard code:

```
<div class="container border mt-5" id="foodItemsContainer" style="display: None;">
  <h2 class="mt-4">All Food Items</h2>
  <table class="table table-striped table-bordered mt-3">
    <thead class="thead-dark">
      <tr>
        <th scope="col">#</th>
        <th scope="col">Name</th>
        <th scope="col">Description</th>
        <th scope="col">Price</th>
      </tr>
    </thead>
    <tbody>
      {% for item in all_items %}
        <tr>
          <th scope="row">{{ forloop.counter }}</th>
          <td>{{ item.name }}</td>
          <td>{{ item.description }}</td>
          <td>${{ item.price }}</td>
        </tr>
      {% endfor %}
    </tbody>
  </table>
</div>
```

13) Adding new items for restaurant:

```
<div class="container border mt-5" id="additemformContainer" style="display: None;">
  <div class="form-group">
    <form action="{% url 'dashboard' %}" method="post">
      {% csrf_token %}
      {{ form|crispy }}
      <button type="submit" class="btn btn-primary">Submit</button>
    </form>
  </div>
</div>
<script>
```

14) Dynamically changing the container using Javascript

```
88 <script>
89     document.getElementById('showMenuButton').addEventListener('click', function() {
90         var foodItemsContainer = document.getElementById('foodItemsContainer');
91         if (foodItemsContainer.style.display === 'none') {
92             foodItemsContainer.style.display = 'block';
93             additemformContainer.style.display = 'none';
94             ordersContainer.style.display = 'none';
95         } else {
96             foodItemsContainer.style.display = 'none';
97         }
98     });
99
100     document.getElementById('additemform').addEventListener('click', function() {
101         var additemformContainer = document.getElementById('additemformContainer');
102         if (additemformContainer.style.display === 'none') {
103             additemformContainer.style.display = 'block';
104             foodItemsContainer.style.display = 'none';
105             ordersContainer.style.display = 'none';
106         } else {
107             additemformContainer.style.display = 'none';
108         }
109     });
110
111     document.getElementById('showorderform').addEventListener('click', function() {
112         var ordersContainer = document.getElementById('ordersContainer');
113         if (ordersContainer.style.display === 'none') {
114             ordersContainer.style.display = 'block';
115             foodItemsContainer.style.display = 'none';
116             additemformContainer.style.display = 'none';
117         } else {
118             ordersContainer.style.display = 'none';
119         }
120     });
121 </script>
122 {% endblock %}
```

15) Showing order for the restaurant owner

```
<div class="container border mt-5" id="ordersContainer" style="display: none;">
  <h2 class="mt-4">All Orders</h2>
  {% if orders %}
    <table class="table table-striped table-bordered mt-3">
      <thead class="thead-dark">
        <tr>
          <th scope="col">Order #</th>
          <th scope="col">Total Amount</th>
          <th scope="col">Status</th>
          <th scope="col">Action</th>
        </tr>
      </thead>
      <tbody>
        {% for order in orders %}
          <tr>
            <td>{{ order.id }}</td>
            <td>${{ order.total_amount }}</td>
            <td>{{ order.status }}</td>
            <td>
              {% if order.status != 'COMPLETED' %}
                <form action="{% url 'complete_order' order.id %}" method="post" style="display: inline-block;">
                  {{ csrf_token }}
                  <button type="submit" class="btn btn-success btn-sm">Complete</button>
                </form>
              {% else %}
                Completed
              {% endif %}
            </td>
          </tr>
        {% endfor %}
      </tbody>
    </table>
  {% else %}
    </table>
    {% else %}
      <div class="alert alert-info" role="alert">
        You have no orders yet.
      </div>
    {% endif %}
  </div>
```

16) Buyer cart

```
1  {% extends 'dashboard/base_dashboard.html' %}
2
3  {% block content %}
4      <div class="container mt-5">
5          <h1 class="mb-4">Cart</h1>
6          <div class="row">
7              <div class="col-md-8">
8                  <div class="card">
9                      <div class="card-header">
10                         <h5 class="mb-0">Cart Items</h5>
11                     </div>
12                     <div class="card-body">
13                         {% if items %}
14                             <ul class="list-group">
15                                 {% for item in items %}
16                                     <li class="list-group-item d-flex justify-content-between align-items-center">
17                                         <div>
18                                             <strong>{{ item.food_item.name }}</strong><br>
19                                             Quantity: {{ item.quantity }}<br>
20                                             Price: ${{ item.get_total_price }}
21                                         </div>
22                                         <form method="post" action="{% url 'remove_from_cart' %}" style="margin: 0;">
23                                             {% csrf_token %}
24                                             <input type="hidden" name="cart_item_id" value="{{ item.id }}">
25                                             <button type="submit" class="btn btn-danger btn-sm">Remove</button>
26                                         </form>
27                                     </li>
28                                 {% endfor %}
29                             </ul>
30                         {% else %}
31                             <div class="alert alert-info" role="alert">
32                                 Your cart is empty.
33                             </div>
34                         {% endif %}
35                     </div>
36                     <div class="card-footer text-end">
37                         <!--<h4>Total: ${{ total }}</h4> -->
38                     </div>
39                 </div>
40             </div>
41             <div class="col-md-4 d-flex flex-column">
42                 {% if restaurant_name %}
43                     <a href="{% url 'restaurant_menu' user_name=user.name restaurant_name=restaurant_name %}" class="btn btn-primary btn-lg mb-2">
44                         <i class="bi bi-cart-plus"></i> Add more items
45                     </a>
46                 {% else %}
47                     <a href="{% url 'dashboard' %}" class="btn btn-primary btn-lg mb-2">
48                         <i class="bi bi-cart-plus"></i> Add more items
49                     </a>
50                 {% endif %}
51
52                 <button class="btn btn-success btn-lg id="viewOrderSummaryBtn">
53                     <i class="bi bi-check-circle"></i> Review Order
54                 </button>
55             </div>
56         </div>
57     </div>
```

17) Buyer review order & placement

```

60 <div class="modal fade" id="orderSummaryModal" tabindex="-1" aria-labelledby="orderSummaryLabel" aria-hidden="true">
61   <div class="modal-dialog">
62     <div class="modal-content">
63       <div class="modal-header">
64         <h5 class="modal-title" id="orderSummaryLabel">Order Summary</h5>
65         <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
66       </div>
67       <div class="modal-body">
68         <div id="orderSummaryContent">
69
70         </div>
71       </div>
72       <div class="modal-footer">
73         <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Close</button>
74         <a href="{% url 'place_order' %}" class="btn btn-primary">Place Order</a>
75       </div>
76     </div>
77   </div>
78 </div>

```

```

80 <script>
81   document.getElementById('viewOrderSummaryBtn').addEventListener('click', function() {
82     fetch("{% url 'get_order_summary' %}")
83       .then(response => response.json())
84       .then(data => {
85         let orderSummary = '<ul class="list-group">';
86         data.items.forEach(item => {
87           orderSummary += `
88             <li class="list-group-item">
89               <strong>${item.food_item}</strong><br>
90               Quantity: ${item.quantity}<br>
91               Total: ${item.total_price}
92             </li>
93           `;
94         });
95         orderSummary += '</ul>';
96         orderSummary += `<h4 class="text-end">Total: ${data.total}</h4>`;
97         document.getElementById('orderSummaryContent').innerHTML = orderSummary;
98         var orderSummaryModal = new bootstrap.Modal(document.getElementById('orderSummaryModal'));
99         orderSummaryModal.show();
100       });
101   });
102 </script>
103 {% endblock %}
104

```


18) Forget password for any user

```
1 {% extends 'members/base.html' %}
2 {% load crispy_forms_tags %}
3 {% block content %}
4 <div class="container mt-5">
5     <div class="row justify-content-center">
6         <div class="col-md-6">
7             <div class="card shadow-sm">
8                 <div class="card-body">
9                     <h2 class="text-center mb-4">Reset Password</h2>
10                    <form method="POST">
11                        {% csrf_token %}
12                        <div class="form-group mb-3">
13                            {{ form.email.label_tag }}
14                            {{ form.email }}
15                            {{ form.email.errors }}
16                        </div>
17                        <div class="form-group mb-3">
18                            {{ form.new_password.label_tag }}
19                            {{ form.new_password }}
20                            {{ form.new_password.errors }}
21                        </div>
22                        <div class="form-group mb-3">
23                            {{ form.confirm_password.label_tag }}
24                            {{ form.confirm_password }}
25                            {{ form.confirm_password.errors }}
26                        </div>
27                        <button type="submit" class="btn btn-primary btn-block">Reset Password</button>
28                    </form>
29                </div>
30            </div>
31        </div>
32    </div>
33 </div>
34 {% endblock %}
```

Backend Development

1)Following code is for landing page

```
import random
from django.shortcuts import render, redirect
from .models import FoodItem, Buyer, Restaurant, Deliveryman
from .forms import BuyerRegisterForm, RestaurantRegisterForm, DeliveryManRegisterForm, ForgetPasswordForm
from django.contrib import messages

def register_user(request):
    if request.method == "POST":
        form = BuyerRegisterForm(request.POST)
        if form.is_valid():
            form.save()
            messages.success(request, 'Registration successful!')
            return redirect('register_user')
        else:
            messages.error(request, 'Registration failed. Please correct the errors in the form. Thank You.')
    else:
        form = BuyerRegisterForm()

    cities = {
        'Dhaka': 'members/dhaka.jpg',
        'Chittagong': 'members/chittagon.jpg',
        'Bagerhat': 'members/bagerhat.jpg',
        'Bandarban': 'members/bandarban.jpeg',
        'Barguna': 'members/barguna.webp',
        'Barishal': 'members/barishal.jpg',
        'Chadpur': 'members/chadpur.jpeg',
        'Bogra': 'members/bogra.jpg',
        'Brahmanbaria': 'members/brahmanbaria.jpg',
    }

    context = {
        'form': form,
        'cities': cities
    }
    return render(request, 'members/landing.html', context)
```

2)Following code is for Buyer/Restaurant Owner/Delivery Man registration.

```
if request.method == "POST":
    form = BuyerRegisterForm(request.POST)
    if form.is_valid():
        form.save()
        messages.success(request, 'Registration successful!')
        return redirect('register_user')
    else:
        messages.error(request, 'Registration failed. Please correct the errors in the form. Thank You.')
else:
    form = BuyerRegisterForm()
```

```

def register_restaurant_user(request):
    if request.method == "POST":
        form = RestaurantRegisterForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('register_user')
    else:
        form = RestaurantRegisterForm()
    return render(request, 'members/restaurant_reg.html' , {"form":form})

def delivery_registration(request):
    if request.method == "POST":
        form = DeliveryManRegisterForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('register_user')
    else:
        form = DeliveryManRegisterForm()
    return render(request, 'members/deliveryman_reg.html' , {"form":form})

```

3)Following code is for Buyer/Restaurant Owner/Delivery Man Login

```

def login_user(request):
    if request.method == "POST":
        email = request.POST['email']
        password = request.POST['password']
        user, user_type = authenticate_user(email, password)
        if user is not None:
            request.session['user_type'] = user_type
            request.session['user_email'] = user.email
            request.session['user_name'] = user.name
            return redirect('dashboard')
        else:
            messages.error(request, "Invalid email or password")
            return redirect('register_user')

    return render(request, 'members/landing.html')

```

Login uses authentication

```
def authenticate_user(email, password):
    try:
        buyer = Buyer.objects.get(email=email)
        if buyer.password == password:
            return buyer, 'Buyer'
    except Buyer.DoesNotExist:
        pass
    try:
        restaurant = Restaurant.objects.get(email=email)
        if restaurant.password == password:
            return restaurant, 'Restaurant'
    except Restaurant.DoesNotExist:
        pass
    try:
        deliveryman = Deliveryman.objects.get(email=email)
        if deliveryman.password == password:
            return deliveryman, 'Deliveryman'
    except Deliveryman.DoesNotExist:
        pass
    return None, None
```

5)Following code is for forget password feature

```
def forget_password(request):
    if request.method == 'POST':
        form = ForgetPasswordForm(request.POST)
        if form.is_valid():
            email = form.cleaned_data['email']
            new_password = form.cleaned_data['new_password']
            user = None
            for model in [Buyer, Restaurant, Deliveryman]:
                if model.objects.filter(email=email).exists():
                    user = model.objects.get(email=email)
                    break
            if user:
                user.password = new_password
                user.save()
                return redirect('register_user')
        else:
            form = ForgetPasswordForm()

    return render(request, 'members/forget_password.html', {'form': form})
```

6)Following code is for Buyer Dashboard & search functionality

```
def dashboard_view(request):
    user_type = request.session.get('user_type')
    user_name = request.session.get('user_name')
    user_email = request.session.get('user_email')

    if user_type == 'Buyer':
        all_restaurants = list(Restaurant.objects.values_list('name', flat=True))
        selected_restaurants = random.sample(all_restaurants, 4) if len(all_restaurants) >= 3 else all_restaurants
        request.session['selected_restaurants'] = selected_restaurants

        restaurant_items = {}
        for restaurant_name in selected_restaurants:
            items = FoodItem.objects.filter(restaurant_name__exact=restaurant_name)[:5]
            restaurant_items[restaurant_name] = items

        if request.method == 'POST':
            form = RestaurantSearchForm(request.POST)
            if form.is_valid():
                search_query = form.cleaned_data['restaurant_name']

                try:
                    restaurant = Restaurant.objects.get(name__iexact=search_query)
                    return redirect('restaurant_menu', user_name=user_name, restaurant_name=restaurant.name)
                except Restaurant.DoesNotExist:
                    matching_items = FoodItem.objects.filter(name__icontains=search_query)
                    restaurant_items = {}
                    for item in matching_items:
                        restaurant_name = item.restaurant.name
                        if restaurant_name not in restaurant_items:
                            restaurant_items[restaurant_name] = []
                            restaurant_items[restaurant_name].append(item)
                    context = {
                        'restaurant_items': restaurant_items,
                        'user_name': user_name,
                        'user_email': user_email,
                        'form': form,
                        'search_query': search_query,
                    }
                    return render(request, 'dashboard/food.html', context)
            else:
                form = RestaurantSearchForm()

        context = {
            'restaurant_items': restaurant_items,
            'user_name': user_name,
            'user_email': user_email,
            'all_restaurants': all_restaurants,
            'form': form,
        }
        return render(request, 'dashboard/buyer_dashboard.html', context)
```

7)Following code is for Restaurant Dashboard

```
elif user_type == 'Restaurant':
    user_type = request.session.get('user_type')
    user_name = request.session.get('user_name')
    user_email = request.session.get('user_email')
    current_restaurant = Restaurant.objects.get(email=user_email)
    all_items = FoodItem.objects.filter(restaurant=current_restaurant)
    form = AddFooditemsForm(request.POST or None)
    if request.method == "POST" and form.is_valid():
        food_item = form.save(commit=False)
        food_item.restaurant = current_restaurant
        food_item.save()
        return redirect('dashboard')
    else:
        form = AddFooditemsForm()
    context = {
        'user_name' : user_name,
        'user_email' : user_email,
        'all_items' : all_items,
        'description': current_restaurant.description,
        'form':form
    }
    return render(request, 'dashboard/restaurant_dashboard.html', context)
```

8)Following code is for Deliver personnel dashboard

```
elif user_type == 'Deliveryman':
    user_type = request.session.get('user_type')
    user_name = request.session.get('user_name')
    user_email = request.session.get('user_email')
    context = {
        'user_name' : user_name,
        'user_email' : user_email
    }
    return render(request, 'dashboard/deliveryman_dashboard.html', context)
```

9)Following code is for profile updates for any user.

```
def profile(request):
    user_type = request.session.get('user_type')
    user_email = request.session.get('user_email')
    user = None
    if user_type == 'Buyer':
        user = get_object_or_404(Buyer, email=user_email)
        form_class = BuyerRegisterForm
    elif user_type == 'Restaurant':
        user = get_object_or_404(Restaurant, email=user_email)
        form_class = RestaurantRegisterForm
    elif user_type == 'Deliveryman':
        user = get_object_or_404(Deliveryman, email=user_email)
        form_class = DeliveryManRegisterForm

    if request.method == 'POST':
        form = form_class(request.POST, instance=user)
        if form.is_valid():
            form.save()
            request.session['user_type'] = user_type
            request.session['user_email'] = user.email
            request.session['user_name'] = user.name
            messages.success(request, "Your profile has been updated.")
            return redirect('dashboard')
        else:
            messages.error(request, "Please correct the error below.")
    else:
        form = form_class(instance=user)

    context = {
        'user': user,
        'user_name': user.name,
        'form': form
    }
    return render(request, 'dashboard/user_profile.html', context)
```

10)Following code is for showing items in the menu (Line 65 and “all items” key is used to pass data).

```
60 elif user_type == 'Restaurant':
61     user_type = request.session.get('user_type')
62     user_name = request.session.get('user_name')
63     user_email = request.session.get('user_email')
64     current_restaurant = Restaurant.objects.get(email=user_email)
65     all_items = FoodItem.objects.filter(restaurant=current_restaurant)
66     form = AddFooditemsForm(request.POST or None)
67     if request.method == "POST" and form.is_valid():
68         food_item = form.save(commit=False)
69         food_item.restaurant = current_restaurant
70         food_item.save()
71         return redirect('dashboard')
72     else:
73         form = AddFooditemsForm()
74         context = {
75             'user_name': user_name,
76             'user_email': user_email,
77             'all_items': all_items,
78             'description': current_restaurant.description,
79             'form': form
80         }
81     return render(request, 'dashboard/restaurant_dashboard.html', context)
82
```

11) Following code used for adding new items in the menu.

```
form = AddFooditemsForm(request.POST or None)
if request.method == "POST" and form.is_valid():
    food_item = form.save(commit=False)
    food_item.restaurant = current_restaurant
    food_item.save()
    return redirect('dashboard')
else:
    form = AddFooditemsForm()
```

12) User cart code includes (Add, remove and show items)

```
def view_cart(request):
    user = get_object_or_404(Buyer, email=request.session.get('user_email'))
    cart = Cart.objects.filter(user=user).first()

    restaurant_name=None
    if cart and cart.items.exists():
        first_item = cart.items.first()
        if first_item:
            restaurant_name = first_item.food_item.restaurant.name

    if cart is None:
        return render(request, 'dashboard/view-cart.html', {
            'user': None,
            'cart': None,
            'items': [],
            'restaurant_name': None,
        })

    context = {
        'user': user,
        'cart': cart,
        'items': cart.items.all() if cart else [],
        'restaurant_name': restaurant_name
    }
    return render(request, 'dashboard/view-cart.html', context)
```



```

def add_to_cart(request):
    if request.method == 'POST':
        food_item_id = request.POST.get('food_item_id')
        quantity = request.POST.get('quantity')

        if food_item_id and quantity:
            try:
                food_item = get_object_or_404(FoodItem, id=food_item_id)
                user = get_object_or_404(Buyer, email=request.session.get('user_email'))

                cart, created = Cart.objects.get_or_create(user=user)
                cart.add_item(food_item, int(quantity))

                return redirect('view-cart')
            except Exception as e:
                print(f"Error adding to cart: {e}")

    return redirect('view-cart')

```

```

def remove_from_cart(request):
    if request.method == 'POST':
        cart_item_id = request.POST.get('cart_item_id')

        if cart_item_id:
            try:
                cart_item = get_object_or_404(CartItem, id=cart_item_id)
                carts = Cart.objects.filter(items=cart_item)

                for cart in carts:
                    if cart.items.filter(id=cart_item_id).exists():
                        cart.items.remove(cart_item)
                    cart_item.delete()

            except Exception as e:
                print(f"Error removing from cart: {e}")

    return redirect('view-cart')

```

13) Placement of order for buyer

```

def place_order(request):
    user = get_object_or_404(Buyer, email=request.session.get('user_email'))

    cart = Cart.objects.filter(user=user).first()

    if cart is None or cart.items.count() == 0:
        return redirect('view-cart')

    first_cart_item = cart.items.first()
    restaurant = first_cart_item.food_item.restaurant

    order = Order.objects.create(
        restaurant=restaurant,
        buyer=user,
        status='PENDING'
    )

    order_items = []
    total_amount = 0
    for cart_item in cart.items.all():
        order_item = OrderItem.objects.create(
            food_item=cart_item.food_item,
            quantity=cart_item.quantity,
            price=cart_item.food_item.price
        )
        order_items.append(order_item)
        total_amount += order_item.get_total_price()

    order.order_items.set(order_items)

```

```

order.order_items.set(order_items)

order.total_amount = total_amount
order.save()

cart.items.all().delete()

return redirect('order_success')

```

14) Get summary and order success message for buyer order

```

def get_order_summery(request):
    user = get_object_or_404(Buyer, email=request.session.get('user_email'))
    cart = Cart.objects.filter(user=user).first()
    if not cart:
        return JsonResponse({
            'items': [],
            'total': 0
        })

    items = cart.items.all()
    order_details = []

    for item in items:
        order_details.append({
            'food_item': item.food_item.name,
            'quantity': item.quantity,
            'total_price': item.get_total_price()
        })

    return JsonResponse({
        'items': order_details,
        'total': cart.get_total()
    })

def order_success(request):
    return render(request, 'dashboard/order_success.html')

```

15) Viewing received orders for restaurant owner

```

def view_orders(request):
    buyer = get_object_or_404(Buyer, email=request.session.get('user_email'))

    orders = Order.objects.filter(buyer=buyer).order_by('-created_on')

    return render(request, 'dashboard/orders.html', {'orders': orders})

def complete_order(request, order_id):
    order = get_object_or_404(Order, id=order_id)
    order.status = 'COMPLETED'
    order.save()
    return redirect('dashboard')

```

Forms Used in the project:

1) For adding more items in menu by restaurant owner and also form used to search Restaurants and food

```
from django import forms
from members.models import FoodItem

class AddFooditemsForm(forms.ModelForm):
    class Meta:
        model = FoodItem
        fields = ['name', 'price', 'description']

class RestaurantSearchForm(forms.Form):
    restaurant_name = forms.CharField(label='Restaurant Name', max_length=100)
```

2) Adding items to cart form

```
class AddToCartForm(forms.Form):
    food_item_id = forms.IntegerField()
    quantity = forms.IntegerField(min_value=1)
```

3) All user registration form

```
from .models import Buyer, Restaurant, Deliveryman
from django import forms
from django.core.validators import MinLengthValidator, RegexValidator

class BuyerRegisterForm(forms.ModelForm):
    class Meta:
        model = Buyer
        fields = "__all__"

class RestaurantRegisterForm(forms.ModelForm):
    class Meta:
        model = Restaurant
        exclude = ['rating']

class DeliveryManRegisterForm(forms.ModelForm):
    class Meta:
        model = Deliveryman
        fields = "__all__"
```

4) Forget password form

```
class ForgetPasswordForm(forms.Form):
    email = forms.EmailField()
    new_password = forms.CharField(widget=forms.PasswordInput(), validators=[
        MinLengthValidator(8),
        RegexValidator(
            regex='^(?=.*[A-Za-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$',
            message='Password must be at least 8 characters long and include letters, numbers, and special characters'
        )
    ])
    confirm_password = forms.CharField(widget=forms.PasswordInput())

    def clean(self):
        cleaned_data = super().clean()
        new_password = cleaned_data.get('new_password')
        confirm_password = cleaned_data.get('confirm_password')

        if new_password != confirm_password:
            raise forms.ValidationError("Passwords do not match.")

        return cleaned_data
```

Models Used in the Project:

Buyer:

```
from django.db import models
from dashboard.models import Review
from django.core.validators import MinLengthValidator, RegexValidator
class Buyer(models.Model):
    name = models.CharField(max_length=200)
    email = models.EmailField(max_length=200, unique=True)
    phone_number = models.CharField(max_length=20, unique=True)
    location = models.CharField(max_length=255)
    password = models.CharField(max_length=128, validators=[
        MinLengthValidator(8),
        RegexValidator(
            regex='^(?=.*[A-Za-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$',
            message='Password must be at least 8 characters long and include letters, numbers, and special characters.'
        )
    ])
    account_balance = models.DecimalField(max_digits=10, decimal_places=2)
    def __str__(self):
        return self.name
```

Restaurant:

```

class Restaurant(models.Model):
    name = models.CharField(max_length=200)
    email = models.EmailField(max_length=200, unique=True)
    phone_number = models.CharField(max_length=20, unique=True)
    location = models.CharField(max_length=255)
    password = models.CharField(max_length=128, validators=[
        MinLengthValidator(8),
        RegexValidator(
            regex='^(?=.*[A-Za-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$',
            message='Password must be at least 8 characters long and include letters, numbers, and special characters.'
        )
    ])
    description = models.TextField()
    rating = models.DecimalField(max_digits=3, decimal_places=1)
    def __str__(self):
        return self.name

```

DeliveryMan & Food items

```

class Deliveryman(models.Model):
    name = models.CharField(max_length=200)
    email = models.EmailField(max_length=200, unique=True)
    phone_number = models.CharField(max_length=20, unique=True)
    location = models.CharField(max_length=255)
    password = models.CharField(max_length=128, validators=[
        MinLengthValidator(8),
        RegexValidator(
            regex='^(?=.*[A-Za-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$',
            message='Password must be at least 8 characters long and include letters, numbers, and special characters.'
        )
    ])
    vehicle_number = models.CharField(max_length=20)
    def __str__(self):
        return self.name

class FoodItem(models.Model):
    restaurant = models.ForeignKey(Restaurant, on_delete=models.CASCADE)
    name = models.CharField(max_length=200)
    description = models.TextField()
    price = models.DecimalField(max_digits=10, decimal_places=2)

    def __str__(self):
        return self.name

```

Reviews for restaurant, CartItem and Cart Model

```

from django.db import models

class Review(models.Model):
    restaurant = models.ForeignKey('members.Restaurant', on_delete=models.CASCADE)
    buyer = models.ForeignKey('members.Buyer', on_delete=models.CASCADE)
    rating = models.IntegerField()
    review_message = models.TextField()

    def __str__(self):
        return f"Review by {self.buyer} for {self.restaurant}"

class CartItem(models.Model):
    user = models.ForeignKey('members.Buyer', on_delete=models.CASCADE)
    food_item = models.ForeignKey('members.FoodItem', on_delete=models.CASCADE)
    quantity = models.PositiveIntegerField(default=1)

    def __str__(self):
        return f"{self.quantity} x {self.food_item.name}"

    def get_total_price(self):
        return self.quantity * self.food_item.price

class Cart(models.Model):
    user = models.OneToOneField('members.Buyer', on_delete=models.CASCADE)
    items = models.ManyToManyField(CartItem)

    def __str__(self):
        return f"Cart for {self.user.name}"

    def get_total(self):
        return sum(item.get_total_price() for item in self.items.all())

```

Food Order

```
class OrderItem(models.Model):
    food_item = models.ForeignKey('members.FoodItem', on_delete=models.CASCADE)
    quantity = models.PositiveIntegerField()
    price = models.DecimalField(max_digits=10, decimal_places=2)

    def __str__(self):
        return f"{self.quantity} x {self.food_item.name}"

    def get_total_price(self):
        return self.quantity * self.price

class Order(models.Model):
    restaurant = models.ForeignKey('members.Restaurant', on_delete=models.CASCADE)
    buyer = models.ForeignKey('members.Buyer', on_delete=models.CASCADE)
    created_on = models.DateTimeField(auto_now_add=True)
    order_items = models.ManyToManyField(OrderItem)
    total_amount = models.DecimalField(max_digits=10, decimal_places=2, blank=True, null=True)
    status_choices = [
        ('PENDING', 'Pending'),
        ('CONFIRMED', 'Confirmed'),
        ('IN_PROGRESS', 'In Progress'),
        ('DELIVERED', 'Delivered'),
        ('CANCELLED', 'Cancelled'),
    ]
    status = models.CharField(max_length=20, choices=status_choices, default='PENDING')

    def __str__(self):
        return f"Order {self.id} by {self.buyer} at {self.restaurant}"

    def calculate_total_amount(self):
        total_amount = sum(item.get_total_price() for item in self.order_items.all())
        self.total_amount = total_amount
        self.save()
        return total_amount
```

Technology(Framework, Languages)

Backend Framework: Django

Frontend Framework: Bootstrap 5

Other Languages: Vanilla CSS, JS

Github Repo Link

Link: <https://github.com/kazirafi09/BiteExpress471FinalProject.git>

Individual Contribution

Id	Name	Contribution
21301381	Kazi Rafiul Kader	Landing Page Creation, Account Creation, Login, Password Reset, Search Functionality
21201402	Nazmus Sakib	Browsing the Menu, Order Placement
19101560	Faiyaaz Ismail	Profile Management, Cart Management
21101015	Tasnim Hasan Rosan	Order Customization, Menu Management